

Interaction avec des sources externes

Module 7 — Fichiers, S3, JDBC, Kafka et Delta Lake

Objectifs pédagogiques

- * Lire et écrire des fichiers locaux dans les formats courants (CSV, JSON, Parquet)
- * Connecter Spark à Amazon S3 — et aux équivalents GCS et Azure Blob Storage
- * Interroger une base de données relationnelle via JDBC (PostgreSQL, MySQL, Snowflake...)
- * Comprendre les bases de Spark Structured Streaming avec Kafka
- * Appréhender le rôle de Delta Lake comme couche transactionnelle sur Parquet

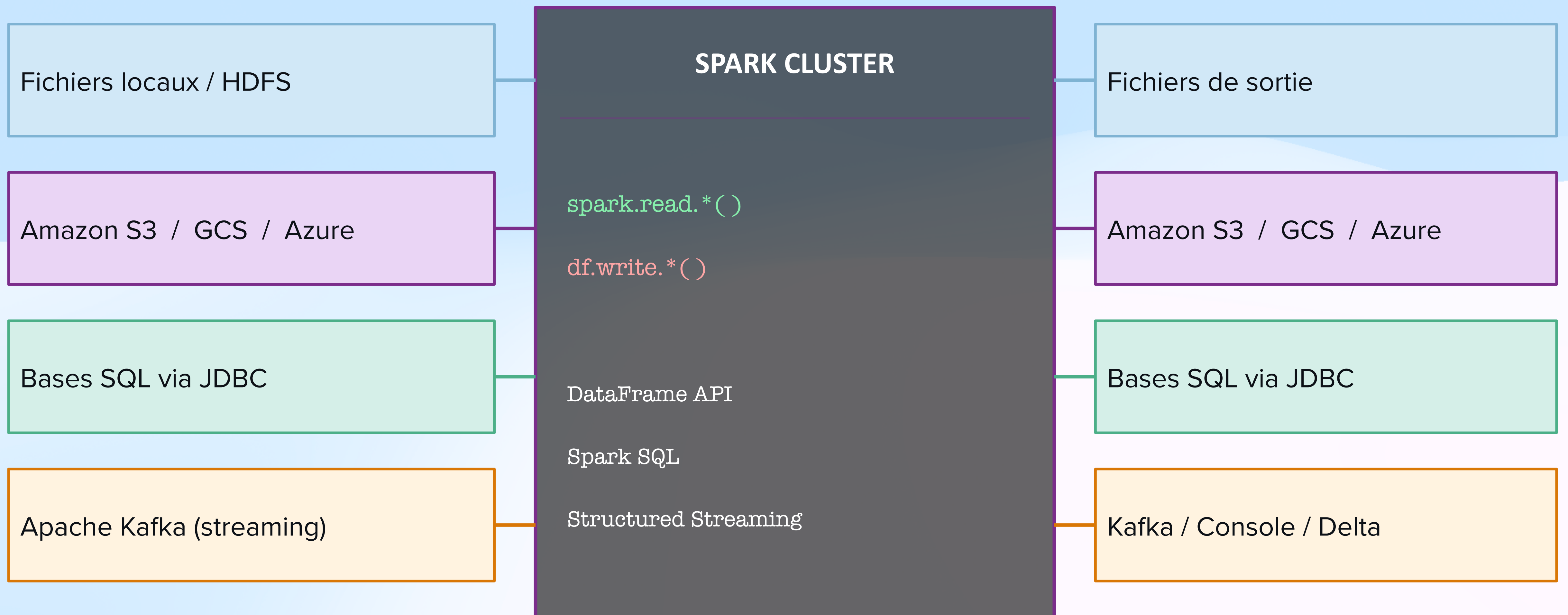
Une API unifiée

L'API unifiée — `spark.read` / `df.write`

`spark.read` retourne toujours un DataFrame, peu importe la source.

`df.write` accepte n'importe quel format ou destination. La même chaîne de transformations s'applique à toutes les sources.

L'API unifiée — spark.read / df.write



Fichiers locaux

Structure générale et options communes

Même syntaxe fluide pour tous les formats — seules les options spécifiques varient

Structure générale

```
df = spark.read\  
  .format("csv")\  
  .option("clé", "valeur")\  
  .schema(mon_schema)\  
  .load("chemin/vers/data")
```

Options communes à tous les formats

```
df = spark.read\  
  .option("inferSchema", "true")\  
  .option("nullValue", "NULL")\  
  .option("mode", "DROPMALFORMED")\  
  .load("data/")
```

mode : PERMISSIVE (défaut), DROPMALFORMED, FAILFAST

- * Quel est le format du fichier ?
- * Quelles options de lecture ?
- * Existe-t-il un schéma de données ?
- * Quelle est le format des dates ?
- * Quelle est la source ?

- * Faut-il déduire le schéma de données ?
- * Que faire des valeurs « null » ?
- * Que faire lignes mal formées ?
- * Quel est le fichier cible ?

Lire plusieurs fichiers simultanément

```
spark.read.option("header","true").csv("data/ventes_*.csv") # wildcards  
spark.read.option("header","true").csv("data/2024/", "data/2025/") # plusieurs répertoires  
spark.read.parquet("data/pays=France/", "data/pays=Allemagne/") # Parquet partitionné
```

Structure générale et options communes

inferSchema vs schéma explicite : `inferSchema=true` est pratique pour l'exploration mais effectue un second passage sur les données → lent. En production, toujours fournir un schéma `StructType` explicite pour les meilleures performances et la validation des types.

CSV — Options clefs

```
df = spark.read\  
  .option("header", "true")\  
  .option("sep", ";")\  
  .option("encoding", "UTF-8")\  
  .option("dateFormat", "dd/MM/yyyy")\  
  .csv("data/ventes.csv")
```

Écriture

```
df.write.option("header", "true")\  
  .option("sep", ";")\  
  .mode("overwrite")\  
  .csv("output/ventes_export/")
```

- * Le fichier a-t-il un entête ?
- * Quel est le séparateur de champs ?
- * Quel est le jeu de caractères ?
- * Quelle est le format des dates ?
- * Quel est le fichier source ?

- * Écrire le fichier avec un entête ?
- * Quel est le séparateur de champs ?
- * Faut-il remplacer les données existantes ?
- * Quel est le fichier cible ?

JSON — Options spécifiques

```
# JSON Lines (défaut Spark)  
df = spark.read.json("data/users.json")
```

```
# JSON multiligne  
df = spark.read\  
  .option("multiLine","true")\  
  .json("data/config.json")
```

```
# Accéder aux champs imbriqués  
df.select("nom",  
  F.col("adresse.ville"),  
  F.explode("commandes")  
)
```

* Lecteur spécifique à JSON

* Le fichier est-il multi-ligne ?

N.B. : La lecture de fichiers JSON multi-lignes ne peut pas être parallélisée. Le fichier est traité dans son ensemble.

Schémas explicite de données

Recommandé en production

```
from pyspark.sql.types import StructType, StructField,
from pyspark.sql.types import IntegerType, StringType, DoubleType, DateType

schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("nom", StringType(), True),
    StructField("montant", DoubleType(), True),
    StructField("date", DateType(), True),
])
df = spark.read.schema(schema).option("header","true").csv("data/ventes.csv")
```



Indiquer un schéma explicite permet à Spark de typer automatiquement les colonnes du DataFrame produit

Parquet — Format de référence en production

Avantages clefs

Colonnaire

Ne lit que les colonnes nécessaires → moins d'I/O

Schéma embarqué

Pas d'inferSchema — types exacts toujours préservés

Compression 5-10×

Bien plus compact que CSV — économie de stockage et réseau

Predicate pushdown

Les filtres sont poussés à la lecture — moins de données chargées

Partitionnement

Partition pruning natif — lecture ciblée des sous-dossiers

Lecture d'un fichier Parquet

```
# Lecture (schéma embarqué → rapide)
df = spark.read.parquet("data/transactions/")

# Plusieurs chemins
df = spark.read.parquet("data/2023/", "data/2024/")

# Partition pruning : Spark lit uniquement
# les sous-dossiers pertinents
df = spark.read.parquet("data/ventes/")\
    .filter(F.col("pays")=="France")\
    .filter(F.col("annee")==2024)
# → lit seulement pays=France/annee=2024/
```

Les données au format Parquet
sont dans un dossier qui
renferme aussi le schéma

Il est possible de lire plusieurs
dossiers simultanément

Ne lit que les fichiers pertinents

Écriture d'un fichier Parquet

```
# Écriture Parquet partitionnée
df.write\
  .option("compression","snappy")\
  .partitionBy("pays","annee")\
  .mode("overwrite")\
  .parquet("output/ventes_parquet/")
```

```
# Compressions disponibles
# snappy → rapide, taux moyen
# gzip  → lent, taux élevé
# zstd  → équilibre (Spark 3.x)
# lz4   → très rapide, taux faible
# none  → sans compression
```

- * Le fichier doit-il être compressé ?
- * Comment le fichier doit-il être partitionné ?
- * Doit-on effacer les données existantes ?
- * Quel est le fichier cible ?

N.B. : On indique, non pas le nom d'un fichier, mais l'emplacement d'un dossier : le partitionnement crée en effet une hiérarchie de sous-dossiers

N.B. : À la lecture Spark crée automatiquement des partitions pour permettre l'a parallélisation du traitement des données.

→ À l'écriture, chaque partition est stockée dans un fichier différent, voire dans des sous-dossiers avec l'option `partitionBy`

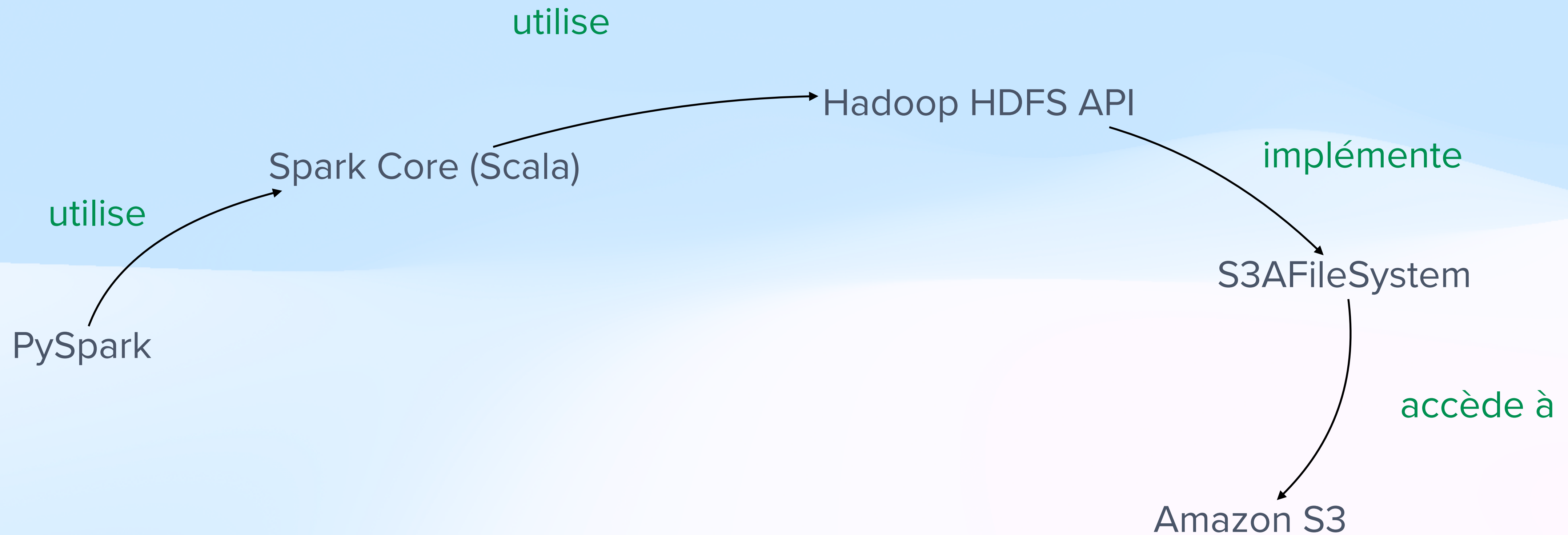
Amazon S3 et le « cloud storage »

Amazon S3 — Configuration

- * **Spark ne parle pas directement à S3** → il utilise l'API FileSystem de Hadoop
- * **hadoop-aws** est le package qui implémente S3AFileSystem
- * **Configuration Hadoop** (`sc._jsc.hadoopConfiguration()`) est utilisée par Spark pour S3
- * **Même en mode local** (`local[*]`), PySpark utilise Hadoop internement

***N.B.** : Cela vaut aussi si vous utilisez un stockage local comme MinIO, qui réplique les buckets S3 d'Amazon*

Amazon S3 — Pipeline



N.B. : Cela vaut aussi si vous utilisez un stockage local comme MinIO, qui réplique les buckets S3 d'Amazon

Amazon S3 — Installation

```
# Installation avec pip  
pip install pyspark
```

```
# Pour accéder à S3, ajouter les packages :  
pyspark --packages org.apache.hadoop:hadoop-aws:3.3.1,com.amazonaws:aws-java-sdk-bundle:1.11.901
```

Il faut fournir à PySpark les interfaces qui permettent au pipeline de fonctionner.

(notez la syntaxe Java, qui est toujours présent en arrière-plan)

Amazon S3 — Configuration (Python)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("S3Example") \
    .master("local[*]") \
    .getOrCreate()

# Configurer les credentials
sc = spark.sparkContext
hadoopConf = sc._jsc.hadoopConfiguration()

hadoopConf.set("fs.s3a.access.key", "YOUR_ACCESS_KEY")
hadoopConf.set("fs.s3a.secret.key", "YOUR_SECRET_KEY")
hadoopConf.set("fs.s3a.endpoint", "s3.eu-west-1.amazonaws.com")
hadoopConf.set("fs.s3a.aws.credentials.provider",
    "com.amazonaws.auth.EnvironmentVariableCredentialsProvider")

# Alternative : utiliser AWS CLI (aws configure)
```

Configuration des clefs
d'accès à Amazon S3
dans un programme
Python



Amazon S3 — Utilisation

```
# Protocoles S3 :  
# - s3:// → ancien (S3 Native)  
# - s3n:// → S3 New (limité à 5TB)  
# - s3a:// → S3 Advanced (recommandé, supporte  
regions V4)  
  
# Lire  
df = spark.read.csv("s3a://my-bucket/data.csv")  
df = spark.read.parquet("s3a://my-bucket/  
data.parquet")  
  
# Écrire en CSV  
df.write\  
.format(« csv »)\  
.option("header", True)\  
save(« s3a://my-bucket/output/« )  
# Écrire en Parquet  
df.write.parquet("s3a://my-bucket/output.parquet")
```

Les mêmes API de lecture/écriture sont utilisées par Spark quel que soit le support.

Amazon S3 — Utilisation avec les rôles IAM

```
# Protocoles S3 :  
# - s3:// → ancien (S3 Native)  
# - s3n:// → S3 New (limité à 5TB)  
# - s3a:// → S3 Advanced (recommandé, supporte  
regions V4)  
  
# Lire  
df = spark.read.csv("s3a://my-bucket/data.csv")  
df = spark.read.parquet("s3a://my-bucket/  
data.parquet")  
  
# Écrire en CSV  
df.write\  
.format(« csv »)\  
.option("header", True)\  
save(« s3a://my-bucket/output/« )  
# Écrire en Parquet  
df.write.parquet("s3a://my-bucket/output.parquet")
```

Les mêmes API de lecture/écriture sont utilisées par Spark quel que soit le support.



Amazon S3 — Utilisation avec les rôles IAM

Pas de secret à gérer

Nécessite la configuration d'un utilisateur et de rôles sur la plateforme d'Amazon + **aws** en ligne de commande

Plutôt que d'utiliser des identifiants, une solution plus simple et plus sûre consiste à utiliser les rôles IAM d'Amazon via la bibliothèque **boto3**.

Fonctionne localement avec MinIO sans avoir besoin des rôles IAM

Amazon S3 — Protocoles et credentials

Spark accède à S3 via le connecteur `hadoop-aws` — protocole `s3a://` recommandé
Les 3 protocoles S3 Credentials — 4 méthodes

1 — Variables d'environnement

```
AWS_ACCESS_KEY_ID +  
AWS_SECRET_ACCESS_KEY
```

*recommandée en
local*

2 — Code Python

```
hadoop_conf.set( 'fs.s3a.access.key' ,  
... )
```

*jamais en
production !*

3 — IAM Role (EC2/EMR)

```
InstanceProfileCredentialsProvider  
auto
```

*recommandée en
production*

4 — `spark-defaults.conf`

```
spark.hadoop.fs.s3a.access.key ...
```

cluster partagé

Règle d'or sécurité : ne jamais écrire les credentials en dur dans le code source. En production AWS, utiliser les IAM Roles (aucun secret à gérer). Sinon, utiliser AWS Secrets Manager ou HashiCorp Vault et injecter les secrets au runtime.

GCS, Azure Blob Storage et optimisations S3

```
com.google.cloud.hadoop.fs.gcs.  
.GoogleHadoopFileSystem")  
hadoop_conf.set("google.cloud.auth.service
```

```
# S3A Magic Committer (atomique)
```

```
hadoop_conf.set(
```

```
"fs.s3a.committer.magic.enabled",
```

et les credentials changent
Optimisations spécifiques à S3

```
true)
```

```
# Threads de chargement
```

```
hadoop_conf.set("fs.s3a.threads.max", "64")
```

```
hadoop_conf.set("fs.s3a.connection.maximum", "64"  
)
```

```
.account.json.keyfile",
```

```
"/chemin/vers/sa.json")
```

```
df = spark.read.parquet("gs://mon-bucket/data/")
```

```
# Azure Blob — wasbs://
```

Particularité de S3 : S3 est un stockage objet (pas un système de fichiers). Il a une latence de listage élevée — lister des milliers de fichiers peut être lent. Pour atténuer : (1) organiser les données en partitions Parquet, (2) activer le S3A Magic Committer pour les écritures atomiques, (3) augmenter les threads S3A.

```
spark.conf.set("spark.sql.parquet
```

Azure ADLS Gen2 : Protocole **abfss://** (meilleure performance que wasbs://) — supporte OAuth2 avec Service Principal pour l'authentification en production.

```
"wasbs://conteneur@compte.blob
```

```
.core.windows.net/data/")
```

JDBC et l'accès aux bases de données

```
jdbc_url = "jdbc:postgresql://srv:5432/ma_base"
```

JDBC — Lecture depuis une base de données

```
props = {  
  "user": os.environ["DB_USER"],  
  "password":os.environ["DB_PASSWORD"],  
  "driver": "org.postgresql.Driver"  
}
```

❌ Lecture séquentielle (1 partition)

```
df = spark.read.jdbc(  
  url=jdbc_url, table="ventes",  
  properties=props  
)
```

✅ Lecture parallèle (N partitions)

```
df = spark.read.jdbc(  
  url=jdbc_url, table="ventes",  
  column="id",  
  lowerBound=1,
```

Requête SQL personnalisée

```
query = """(SELECT id, nom, montant  
  FROM ventes WHERE annee=2024) AS t"""
```

```
df = spark.read.jdbc(  
  url=jdbc_url, table=query,  
  properties=props)
```

Partitionnement par date

```
df = spark.read.jdbc(  
  url=jdbc_url,  
  table="transactions",  
  column="date_transaction",  
  lowerBound="2024-01-01",  
  upperBound="2024-12-31",  
  numPartitions=12,  
  properties=props)
```

Fonctionnement parallèle : avec numPartitions=20 et id entre 1 et 10M, Spark exécute 20 requêtes SQL concurrentes : WHERE id >= 1 AND id < 500001 / WHERE id >= 500001 AND id < 1000001 ... La colonne de partitionnement doit être **indexée dans la base**

Base de données	Package Maven	Classe driver
PostgreSQL	org.postgresql:postgresql:42.6.0	org.postgresql.Driver
MySQL	com.mysql:mysql-connector-j:8.1.0	com.mysql.cj.jdbc.Driver
SQL Server	com.microsoft.sqlserver:mssql-jdbc:12.4	com.microsoft.sqlserver.jdbc.SQLServerDriver
Snowflake	net.snowflake:spark-snowflake_2.12:2.12.0	net.snowflake.client.jdbc.SnowflakeDriver

```
table="resultats_ml",
```

```
mode="overwrite"
```

```
properties=props
```

```
)
```

```
# Avec options de performance
```

```
# Credentials via secrets manager (AWS)
```

```
df.coalesce(4).write.jdbc(
```

```
import boto3, json
url=jdbc_url,
```

```
client = boto3.client("secretsmanager", region_name="eu-west-1")
```

```
secret = json.loads(client.get_secret_value(SecretId="prod/spark/db")["SecretString"])
```

```
props = {"user":secret["username"],"password":secret["password"],"driver":"org.postgresql.Driver"}
```

```
# Filtrer côté base avant chargement
```

```
*truncate": "false"}
```

```
query = "(SELECT * FROM ventes WHERE annee=2024 AND montant>100) AS t"
```

```
df = spark.read.jdbc(url=jdbc_url, table=query, properties=props)
```

JDBC — Écriture et bonnes pratiques

Bonnes pratiques JDBC

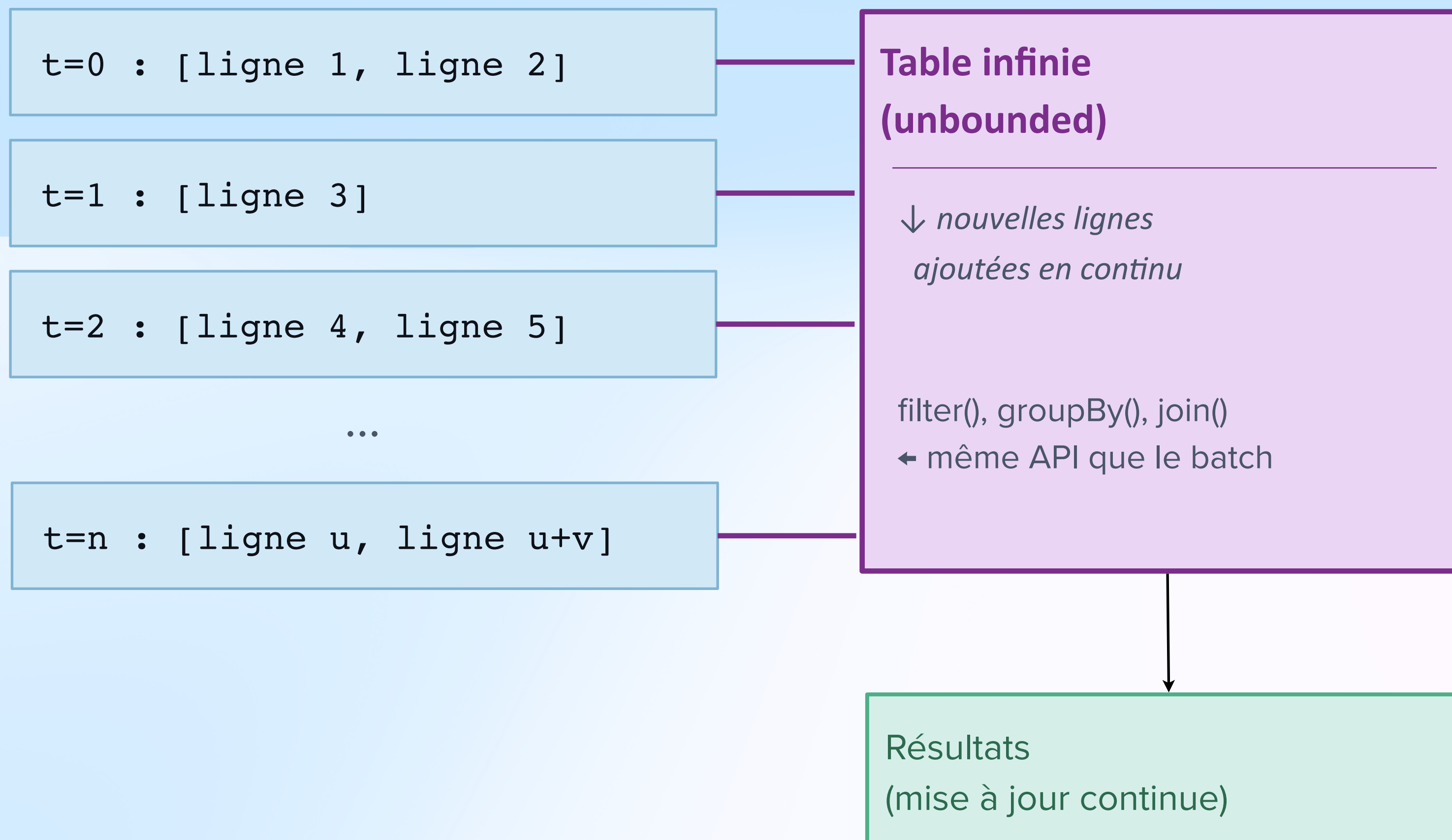
Ne jamais écrire les credentials en dur — utiliser os.environ ou un secrets manager (AWS Secrets Manager, HashiCorp Vault)

- Toujours filtrer côté base avant de charger en Spark (pushdown via sous-requête SQL)
- Indexer la colonne de partitionnement dans la base :
CREATE INDEX idx_id ON ventes(id)
- Réduire le nb de partitions avant écriture (coalesce)

Spark Structured Streaming

Structured streaming : une table infinie

Flux entrant (Kafka, fichiers...)



Modes de déclenchement (trigger)

Trigger.Once()
Traite tout, puis s'arrête

Trigger.AvailableNow()
Idem, multi-batch (3.3+)

ProcessingTime("1 min")
Déclenche toutes les N min

Trigger.Continuous("1s")
Latence ~ms (expérimental)

Structured streaming : une table infinie

Flux entrant (Kafka, fichiers...)

t=0 : [ligne1, ligne2]

t=1 : [ligne3]

t=2 : [ligne4, ligne5]

Table infinie

(unbounded)

↓ nouvelles lignes

ajoutées en continu

filter(), groupBy(), join()

← même API que le batch

Résultats

(mise à jour continue)

Modes de déclenchement (trigger)

Trigger.Once()

Traite tout, puis s'arrête

Trigger.AvailableNow()

Idem, multi-batch (3.3+)

ProcessingTime("1 min")

Déclenche toutes les N min

Trigger.Continuous("1s")

Latence ~ms (expérimental)

Checkpoint obligatoire : le checkpointLocation stocke les offsets Kafka traités, l'état des agrégations et les métadonnées de progression. Sans lui, le stream repart de zéro après un échec. Stocker le checkpoint sur S3 ou HDFS (jamais sur un disque local).

Même API que le batch — un flux est modélisé comme une table sans fin à laquelle de nouvelles lignes s'ajoutent

Exemple de lecture avec Kafka

```
# Dépendance : spark-sql-kafka-0-10_2.12
df_stream = spark.readStream\
    .format("kafka")\
    .option("kafka.bootstrap.servers",
            "kafka:9092")\
    .option("subscribe","topic-ventes")\
    .option("startingOffsets","earliest")\
    .option("maxOffsetsPerTrigger","10000")\
    .load()

# Désérialiser le payload JSON (bytes → struct)
schema_v =
    StructType([StructField("produit",StringType()),
                 StructField("montant",DoubleType()),
                 StructField("ville", StringType())])

df_v = df_stream\
    .select(F.col("value").cast("string").alias("j"))\
    .select(F.from_json(F.col("j"),schema_v).alias("d"))\
    .select("d.*")
```

Spark s'abonne à un topic Kafka, dont le broker est disponible sur le port 9092. Le top écouté est **topic-ventes** et Spark consommera tous les messages envoyés par Kafka.

Sources comprises par Spark

Formats acceptés par défaut par readStream

Format	Usage	Exemple
socket	Flux TCP, test/démo uniquement	nc -lk 9999
rate	Génère des données synthétiques	Tests de performance
json	Fichiers JSON dans un dossier surveillé	Logs applicatifs
csv	Fichiers CSV dans un dossier surveillé	Exports périodiques
parquet	Fichiers Parquet dans un dossier surveillé	Data lake
orc	Fichiers ORC dans un dossier surveillé	Data lake (ecosystème Hive)
text	Fichiers texte brut, une ligne = une row	Logs bruts

Écriture en flux

```
# Agrégation par fenêtre temporelle
df_agg = df_v\
    .withColumn("ts",F.current_timestamp())\
    .groupBy(
        F.window(F.col("ts"),"10 minutes","5 minutes"),
        F.col("ville")
    )\
    .agg(F.sum("montant").alias("ca"),
        F.count("*").alias("nb"))
```

```
# Modes de sortie (output mode)
# "append" → nouvelles lignes seulement
# "complete" → toute la table à chaque batch
# "update" → lignes modifiées seulement
```

```
# Sink console (dev)
q = df_agg.writeStream\
    .outputMode("complete")\
    .format("console")\
    .option("checkpointLocation","/tmp/ckpt/")\
    .start()
```

Des données sont manipulées par Spark

Elles sont ensuite envoyées en flux sur une sortie, ici la console.

checkpointLocation permet de tracer les données déjà consommées et de reprendre le traitement là où il s'était arrêté

Les sorties (« sinks », puits) disponibles

console

Développement et debug — affiche dans le terminal

parquet / csv

Production — écriture continue sur S3 ou HDFS

kafka

Republier vers un autre topic Kafka

foreachBatch

Sink custom — permet d'écrire via JDBC, Delta, etc.

Delta Lake, une couche
transactionnelle

Limites de Parquet

Limites de Parquet seul

- Pas de transactions ACID → écriture partielle peut corrompre
- Pas d'UPDATE / DELETE natif sur les fichiers
- Pas de « time travel » (lire les données à un instant passé)
- Schéma non contrôlé à l'évolution

Opérations ACID

Ajoute un journal de transactions `_delta_log/` aux fichiers Parquet existants

```
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "output/ventes_delta/")

dt.update(condition=F.col("ville")==F.lit("paris"), set={"ville":F.lit("Paris")}) # UPDATE
dt.delete(condition=F.col("montant")<0) # DELETE
dt.alias("e").merge(df_maj.alias("m"), "e.id=m.id")\
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
# MERGE
```

```
output/ventes_delta/
├── _delta_log/
│   ├── 0000...0.json ← CREATE
│   ├── 0000...1.json ← INSERT
│   ├── 0000...2.json ← UPDATE
│   └── ...
├── part-00000-xxxx.parquet
└── part-00001-xxxx.parquet
```

Delta Lake — Time travel et configuration

```
# Time travel — par version
df_v0 = spark.read.format("delta")\
    .option("versionAsOf",0)\
    .load("output/ventes_delta/")

# Time travel — par timestamp
df_hier = spark.read.format("delta")\
    .option("timestampAsOf","2024-01-15 10:00:00")\
    .load("output/ventes_delta/")

# Historique des transactions
dt.history().select(
    "version","timestamp","operation"
).show()
```

```
# Configuration SparkSession
spark = SparkSession.builder\
    .config("spark.jars.packages",
        "io.delta:delta-spark_2.12:3.1.0")\
    .config("spark.sql.extensions",
        "io.delta.sql.DeltaSparkSessionExtension")\
    .config("spark.sql.catalog.spark_catalog",
        "org.apache.spark.sql.delta.catalog"
        ".DeltaCatalog")\
    .getOrCreate()

# Lecture / Écriture
df.write.format("delta").mode("overwrite")\
    .save("output/ventes_delta/")
df = spark.read.format("delta")\
    .load("output/ventes_delta/")
```

Fonctionnalités de Delta Lake

Transactions ACID	Lectures cohérentes même pendant les écritures — plus de données corrompues
UPDATE / DELETE / MERGE	Opérations SQL complètes sur des fichiers Parquet immuables
Time Travel	Lire les données à n'importe quelle version ou timestamp passé
Schéma évolutif	Ajout de colonnes avec contrôle et validation automatique
Optimisation automatique	OPTIMIZE, Z-ORDER, VACUUM pour compacter et nettoyer les petits fichiers

Résumé

Concept	Points clés à retenir
API unifiée	spark.read / df.write fonctionnent identiquement pour tous les formats et sources
Parquet	Format recommandé : schéma embarqué, colonnaire, compressé, predicate pushdown
inferSchema	Pratique mais coûteux — toujours préférer un schéma explicite en production
S3 s3a://	Protocole moderne pour S3 — configurer hadoop-aws et les credentials (jamais en dur)
IAM Role	Méthode recommandée en production AWS — aucun secret à stocker
JDBC parallèle	Spécifier column, lowerBound, upperBound, numPartitions — indexer la colonne côté base
Structured Streaming	Même API que le batch — source → transformations → sink
Checkpoint	Obligatoire pour la reprise après échec — stocker sur S3/HDFS, jamais en local
Mode complete	Réécrit la table entière à chaque batch — nécessite une agrégation
Delta Lake	ACID + UPDATE/DELETE + time travel sur des fichiers Parquet
MERGE	Upsert : whenMatchedUpdateAll() + whenNotMatchedInsertAll() — replace INSERT OR UPDATE

Exercices

Ex. 1 — Formats de fichiers *(25 min)* 50 000 lignes réalistes → écriture CSV, JSON, Parquet snappy et gzip → comparer tailles disque et temps de lecture → vérifier le partition pruning avec explain().

Ex. 2 — Connexion JDBC *(35 min)* PostgreSQL via Docker (docker run postgres) → créer clients avec 100 000 lignes → lire avec 1 puis 8 partitions → comparer temps et plans → écrire une agrégation Spark dans une nouvelle table.

Ex. 3 — Structured Streaming *(40 min)* Répertoire surveillé comme source → script Python générant un nouveau fichier JSON toutes les 5s → stream Spark : CA par ville sur fenêtre de 30s → observer les micro-batches en mode complete.

Ex. 4 — Delta Lake *(30 min)* Écrire en Delta → UPDATE sur une colonne → MERGE avec un DataFrame de mises à jour → afficher l'historique des transactions → lire la version 0 et comparer avec la version courante.

Pour aller plus loin

Documentation

- ✱ Spark Data Sources : spark.apache.org/docs/latest/sql-data-sources.html
- ✱ S3A Guide : hadoop.apache.org/docs/stable/hadoop-aws/tools/hadoop-aws/index.html
- ✱ Structured Streaming : spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- ✱ Delta Lake : docs.delta.io

Outils pour les TPs

MinIO	Équivalent S3 open source pour tester en local — min.io — même API que S3, protocole s3a://
--------------	--

Kafka Docker	docker-compose avec Zookeeper + Kafka + Kafka UI — idéal pour les TPs streaming en local
---------------------	--

Delta Standalone	Lire/écrire Delta sans Spark — delta-standalone pour les applications légères
-------------------------	--